Quarterly Operations Report

A small accessible report that showcases the PDF/UA-2 output of glu's Markdown pipeline. The header `format: PDF/UA-2` in the frontmatter is the only thing that distinguishes this from a plain-PDF render. Every other instruction below is regular CommonMark plus glu's standard extensions.

What this document demonstrates

The file exercises the structural elements that screen readers care about most:

- Heading hierarchy (H1, H2, H3) produces a navigable structure tree with one `h1`, several `h2`, and an `h3` under the HTML5 namespace.
- A table with a header row maps to `table`, `thead`, `tbody`, with `th` cells receiving the appropriate `/Scope` attribute.
- Lists become `ul` plus `li`. Items remain individually selectable.
- A heading-driven outline lets readers jump to any section directly. Under PDF/UA-2 the outline `/Dest` entries are structure destinations rather than page destinations (ISO 14289-2 section 8.8).

Operational metrics

The metrics below are illustrative only.

Quarter	Throughput	Incidents	SLA met
Q1	12 400	3	yes
Q2	14 850	1	yes
Q3	13 720	2	yes
Q4	15 200	0	yes

Each row is wrapped as a `tr` structure element, with the header row inside `thead` and the data rows inside `tbody`.

A note on numbers

The "SLA met" column is a hand-curated assessment, not an auto-derived flag. A real report would link these to the incident-tracker IDs; for this showcase the column is plain text.

Verifying

After running

```
glu accessible-report.md
```

the output is verifiable with veraPDF:

```
verapdf --flavour ua2 accessible-report.pdf
```

The validator reports `isCompliant=true` with zero failed checks. `pdfinfo` confirms `PDF version: 2.0` and `Tagged: yes`.

How structure flows from Markdown to PDF

Each Markdown element maps to a structure role in the HTML5 namespace (lowercase tag names) under PDF/UA-2, or to the corresponding role in a standard PDF namespace when no HTML5 equivalent exists.

- A heading line becomes an `h1` (or `h2`, `h3`, etc.) structure element.
- A list item becomes an `li` inside a `ul`.
- A fenced code block becomes a `p` containing an inline `code`.

No additional CSS or Lua code is needed. The Markdown source tree maps directly to the PDF structure tree, and the HTML5-namespace `/RoleMapNS` declaration links each role back to its PDF Standard Structure Namespace equivalent so assistive technologies see a familiar tag set.